

Сервер аутентификации и векторных вычислений

Создано системой Doxygen 1.9.4

1	Алфавитный указатель пространств имен	1
1.1	Пространства имен	1
2	Алфавитный указатель классов	3
2.1	Классы	3
3	Список файлов	5
3.1	Файлы	5
4	Пространства имен	7
4.1	Пространство имен SHA1	7
4.1.1	Подробное описание	7
5	Классы	9
5.1	Класс AuthManager	9
5.1.1	Подробное описание	9
5.1.2	Конструктор(ы)	10
5.1.2.1	AuthManager()	10
5.1.2.2	~AuthManager()	10
5.1.3	Методы	10
5.1.3.1	addUser()	10
5.1.3.2	authenticate()	10
5.1.3.3	computeSHA1()	10
5.1.3.4	generateSalt64()	10
5.1.3.5	loadUserDatabase()	11
5.1.3.6	padZeros()	11
5.1.3.7	userExists()	11
5.1.4	Данные класса	11
5.1.4.1	users	11
5.2	Класс Database	11
5.2.1	Методы	12
5.2.1.1	add()	12
5.2.1.2	count()	12
5.2.1.3	load()	12
5.2.1.4	verify()	12
5.2.2	Данные класса	12
5.2.2.1	users	12
5.3	Класс Interface	13
5.3.1	Подробное описание	14
5.3.2	Конструктор(ы)	14
5.3.2.1	Interface()	14
5.3.3	Методы	14
5.3.3.1	checkFileExists()	14
5.3.3.2	checkFileWritable()	14
5.3.3.3	getParams()	14

5.3.3.4	<code>getParseErrors()</code>	14
5.3.3.5	<code>isHelpRequested()</code>	15
5.3.3.6	<code>isVersionRequested()</code>	15
5.3.3.7	<code>parse()</code>	15
5.3.3.8	<code>parseOptions()</code>	15
5.3.3.9	<code>printHelp()</code>	15
5.3.3.10	<code>printVersion()</code>	15
5.3.3.11	<code>validateParams()</code>	15
5.3.3.12	<code>validatePort()</code>	16
5.3.4	Данные класса	16
5.3.4.1	<code>helpRequested</code>	16
5.3.4.2	<code>params</code>	16
5.3.4.3	<code>parseErrors</code>	16
5.3.4.4	<code>versionRequested</code>	16
5.4	Класс <code>Logger</code>	16
5.4.1	Подробное описание	17
5.4.2	Конструктор(ы)	17
5.4.2.1	<code>Logger()</code> [1/2]	17
5.4.2.2	<code>Logger()</code> [2/2]	17
5.4.2.3	<code>~Logger()</code>	18
5.4.3	Методы	18
5.4.3.1	<code>getInstance()</code>	18
5.4.3.2	<code>initialize()</code>	18
5.4.3.3	<code>log()</code>	18
5.4.3.4	<code>operator=()</code>	18
5.4.3.5	<code>reset()</code>	18
5.4.4	Данные класса	18
5.4.4.1	<code>instance</code>	19
5.4.4.2	<code>logFile</code>	19
5.5	Структура <code>Params</code>	19
5.5.1	Конструктор(ы)	19
5.5.1.1	<code>Params()</code>	19
5.5.2	Данные класса	19
5.5.2.1	<code>dbFile</code>	20
5.5.2.2	<code>logFile</code>	20
5.5.2.3	<code>port</code>	20
5.6	Класс <code>Server</code>	20
5.6.1	Подробное описание	21
5.6.2	Конструктор(ы)	21
5.6.2.1	<code>Server()</code>	21
5.6.3	Методы	21
5.6.3.1	<code>handleClient()</code>	21
5.6.3.2	<code>receiveText()</code>	22

5.6.3.3 receiveVector()	22
5.6.3.4 run()	22
5.6.3.5 sendResult()	22
5.6.3.6 sendText()	22
5.6.3.7 stop()	22
5.6.4 Данные класса	22
5.6.4.1 auth	23
5.6.4.2 logger	23
5.6.4.3 port	23
5.6.4.4 running	23
5.7 Класс SHA1	23
5.7.1 Методы	23
5.7.1.1 hash() [1/2]	23
5.7.1.2 hash() [2/2]	24
5.8 Класс VectorProcessor	24
5.8.1 Подробное описание	24
5.8.2 Методы	24
5.8.2.1 checkOverflow()	24
5.8.2.2 computeSumOfSquares()	25
5.8.2.3 processVectors()	25
6 Файлы	27
6.1 Файл AuthManager.h	27
6.2 AuthManager.h	28
6.3 Файл SHA1.h	28
6.4 SHA1.h	29
6.5 Файл Logger.h	30
6.5.1 Перечисления	30
6.5.1.1 LogLevel	30
6.6 Logger.h	31
6.7 Файл Server.h	31
6.8 Server.h	32
6.9 Файл VectorProcessor.h	33
6.10 VectorProcessor.h	34
6.11 Файл Interface.h	34
6.12 Interface.h	35
6.13 Файл AuthManager.cpp	36
6.13.1 Подробное описание	36
6.14 Файл database.cpp	36
6.14.1 Функции	37
6.14.1.1 add_user_to_db()	37
6.14.1.2 count_users()	37
6.14.1.3 load_user_database()	37

6.14.1.4 <code>verify_user()</code>	37
6.15 Файл <code>SHA1.cpp</code>	38
6.16 Файл <code>logger.cpp</code>	38
6.17 Файл <code>Logger.cpp</code>	38
6.17.1 Подробное описание	39
6.18 Файл <code>Logger.cpp</code>	39
6.19 Файл <code>main.cpp</code>	40
6.19.1 Подробное описание	40
6.19.2 Функции	40
6.19.2.1 <code>main()</code>	40
6.20 Файл <code>VectorProcessor.cpp</code>	41
6.20.1 Подробное описание	41
6.21 Файл <code>server.cpp</code>	41
6.21.1 Подробное описание	42
6.22 Файл <code>Interface.cpp</code>	42
6.22.1 Подробное описание	43
6.22.2 Макросы	43
6.22.2.1 <code>PROGRAM_NAME</code>	43
6.22.2.2 <code>PROGRAM_VERSION</code>	43
6.22.3 Переменные	43
6.22.3.1 <code>long_options</code>	43
6.22.3.2 <code>short_options</code>	43
Предметный указатель	45

Глава 1

Алфавитный указатель пространств имен

1.1 Пространства имен

Полный список пространств имен.

[SHA1](#)

Функции хеширования SHA-1 [7](#)

Глава 2

Алфавитный указатель классов

2.1 Классы

Классы с их кратким описанием.

AuthManager	
Менеджер аутентификации пользователей	9
Database	11
Interface	
Парсер параметров командной строки	13
Logger	
Потокобезопасный регистратор событий	16
Params	19
Server	
Основной класс сетевого сервера	20
SHA1	23
VectorProcessor	
Обработчик числовых векторов	24

Глава 3

Список файлов

3.1 Файлы

Полный список файлов.

AuthManager.h	27
SHA1.h	28
Logger.h	30
Server.h	31
VectorProcessor.h	33
Interface.h	34
AuthManager.cpp	
Реализация менеджера аутентификации	36
database.cpp	36
SHA1.cpp	38
logger.cpp	38
logger/Logger.cpp	
Реализация логгера	38
Logger/Logger.cpp	39
main.cpp	
Главная функция сервера	40
VectorProcessor.cpp	
Реализация обработчика векторов	41
server.cpp	
Реализация класса Server	41
Interface.cpp	
Реализация парсера командной строки	42

Глава 4

Пространства имен

4.1 Пространство имен SHA1

Функции хеширования SHA-1.

4.1.1 Подробное описание

Функции хеширования SHA-1.

Вычисление хеша от строки и преобразование в шестнадцатеричный формат. Используется для аутентификации клиентов.

Глава 5

Классы

5.1 Класс AuthManager

Менеджер аутентификации пользователей.

```
#include <AuthManager.h>
```

Открытые члены

- `AuthManager()` = default
- `~AuthManager()` = default
- `bool loadUserDatabase (const std::string &filename)`
- `bool userExists (const std::string &username)`
- `bool authenticate (const std::string &username, const std::string &salt, const std::string &client← Hash)`
- `void addUser (const std::string &username, const std::string &passwordHash)`
- `std::string generateSalt64 ()`

Закрытые члены

- `std::string padZeros (const std::string &str, size_t length)`
- `std::string computeSHA1 (const std::string &salt, const std::string &password)`

Закрытые данные

- `std::map< std::string, std::string > users`

5.1.1 Подробное описание

Менеджер аутентификации пользователей.

Загружает базу пользователей из файла, проверяет логины и хеши паролей. Использует схему аутентификации SHA-1 с 64-битной солью от клиента.

5.1.2 Конструктор(ы)

5.1.2.1 AuthManager()

```
AuthManager::AuthManager ( ) [default]
```

5.1.2.2 ~AuthManager()

```
AuthManager::~AuthManager ( ) [default]
```

5.1.3 Методы

5.1.3.1 addUser()

```
void AuthManager::addUser (
    const std::string & username,
    const std::string & passwordHash )
```

5.1.3.2 authenticate()

```
bool AuthManager::authenticate (
    const std::string & username,
    const std::string & salt,
    const std::string & clientHash )
```

5.1.3.3 computeSHA1()

```
std::string AuthManager::computeSHA1 (
    const std::string & salt,
    const std::string & password ) [private]
```

5.1.3.4 generateSalt64()

```
std::string AuthManager::generateSalt64 ( )
```


5.1.3.5 loadUserDatabase()

```
bool AuthManager::loadUserDatabase (
    const std::string & filename )
```

5.1.3.6 padZeros()

```
std::string AuthManager::padZeros (
    const std::string & str,
    size_t length ) [private]
```

5.1.3.7 userExists()

```
bool AuthManager::userExists (
    const std::string & username )
```

5.1.4 Данные класса

5.1.4.1 users

```
std::map<std::string, std::string> AuthManager::users [private]
```

Объявления и описания членов классов находятся в файлах:

- [AuthManager.h](#)
- [AuthManager.cpp](#)

5.2 Класс Database

Открытые члены

- bool [load](#) (const std::string &filename)
- bool [verify](#) (const std::string &user, const std::string &hash)
- size_t [count](#) () const
- void [add](#) (const std::string &user, const std::string &hash)

Закрытые данные

- std::map< std::string, std::string > [users](#)

5.2.1 Методы

5.2.1.1 add()

```
void Database::add (
    const std::string & user,
    const std::string & hash ) [inline]
```

5.2.1.2 count()

```
size_t Database::count ( ) const [inline]
```

5.2.1.3 load()

```
bool Database::load (
    const std::string & filename ) [inline]
```

5.2.1.4 verify()

```
bool Database::verify (
    const std::string & user,
    const std::string & hash ) [inline]
```

5.2.2 Данные класса

5.2.2.1 users

```
std::map<std::string, std::string> Database::users [private]
```

Объявления и описания членов класса находятся в файле:

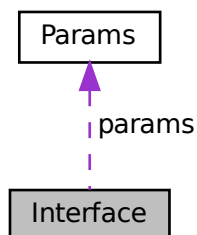
- [database.cpp](#)

5.3 Класс Interface

Парсер параметров командной строки.

```
#include <Interface.h>
```

Граф связей класса Interface:



Открытые члены

- [Interface](#) ()
- bool [parse](#) (int argc, char *argv[])
- [Params](#) [getParams](#) () const
- bool [isHelpRequested](#) () const
- bool [isVersionRequested](#) () const
- std::vector< std::string > [getParseErrors](#) () const
- void [printHelp](#) () const
- void [printVersion](#) () const

Закрытые члены

- void [parseOptions](#) (int argc, char *argv[])
- bool [validateParams](#) ()
- bool [validatePort](#) (int port) const
- bool [checkFileExists](#) (const std::string &filename) const
- bool [checkFileWritable](#) (const std::string &filename) const

Закрытые данные

- [Params](#) [params](#)
- bool [helpRequested](#)
- bool [versionRequested](#)
- std::vector< std::string > [parseErrors](#)

5.3.1 Подробное описание

Парсер параметров командной строки.

Обрабатывает аргументы: `-port`, `-config`, `-log`, `-help`. Валидирует номера портов (1024-65535) и доступность файлов.

5.3.2 Конструктор(ы)

5.3.2.1 Interface()

```
Interface::Interface ( )
```

5.3.3 Методы

5.3.3.1 checkFileExists()

```
bool Interface::checkFileExists (
    const std::string & filename ) const    [private]
```

5.3.3.2 checkFileWritable()

```
bool Interface::checkFileWritable (
    const std::string & filename ) const    [private]
```

5.3.3.3 getParams()

```
Params Interface::getParams ( ) const
```

5.3.3.4 getParseErrors()

```
std::vector< std::string > Interface::getParseErrors ( ) const
```

5.3.3.5 isHelpRequested()

```
bool Interface::isHelpRequested ( ) const
```

5.3.3.6 isVersionRequested()

```
bool Interface::isVersionRequested ( ) const
```

5.3.3.7 parse()

```
bool Interface::parse (
    int argc,
    char * argv[] )
```

5.3.3.8 parseOptions()

```
void Interface::parseOptions (
    int argc,
    char * argv[] ) [private]
```

5.3.3.9 printHelp()

```
void Interface::printHelp ( ) const
```

5.3.3.10 printVersion()

```
void Interface::printVersion ( ) const
```

5.3.3.11 validateParams()

```
bool Interface::validateParams ( ) [private]
```

5.3.3.12 validatePort()

```
bool Interface::validatePort (
    int port ) const    [private]
```

5.3.4 Данные класса

5.3.4.1 helpRequested

```
bool Interface::helpRequested    [private]
```

5.3.4.2 params

```
Params Interface::params    [private]
```

5.3.4.3 parseErrors

```
std::vector<std::string> Interface::parseErrors    [private]
```

5.3.4.4 versionRequested

```
bool Interface::versionRequested    [private]
```

Объявления и описания членов классов находятся в файлах:

- [Interface.h](#)
- [Interface.cpp](#)

5.4 Класс Logger

Потокобезопасный регистратор событий.

```
#include <Logger.h>
```

Граф связей класса Logger:



Открытые члены

- `bool initialize` (`const std::string &filename`)
- `void log` (`LogLevel level`, `const std::string &message`, `const std::string ¶ms=""`)
- `void reset` ()
- `~Logger` ()=default

Открытые статические члены

- `static Logger & getInstance` ()

Закрытые члены

- `Logger` ()
- `Logger` (`const Logger &`)=delete
- `Logger & operator=` (`const Logger &`)=delete

Закрытые данные

- `std::string logFile`

Закрытые статические данные

- `static Logger * instance` = nullptr

5.4.1 Подробное описание

Потокобезопасный регистратор событий.

Записывает сообщения об ошибках и событиях в файл журнала. Добавляет метки времени и уровень критичности.

5.4.2 Конструктор(ы)

5.4.2.1 `Logger()` [1/2]

```
Logger::Logger ( ) [private]
```

5.4.2.2 `Logger()` [2/2]

```
Logger::Logger (
    const Logger & ) [private], [delete]
```

5.4.2.3 ~Logger()

```
Logger::~Logger ( ) [default]
```

5.4.3 Методы

5.4.3.1 getInstance()

```
Logger & Logger::getInstance ( ) [static]
```

5.4.3.2 initialize()

```
bool Logger::initialize (
    const std::string & filename )
```

5.4.3.3 log()

```
void Logger::log (
    LogLevel level,
    const std::string & message,
    const std::string & params = "" )
```

5.4.3.4 operator=()

```
Logger & Logger::operator= (
    const Logger & ) [private], [delete]
```

5.4.3.5 reset()

```
void Logger::reset ( )
```

5.4.4 Данные класса

5.4.4.1 instance

```
Logger * Logger::instance = nullptr [static], [private]
```

5.4.4.2 logFile

```
std::string Logger::logFile [private]
```

Объявления и описания членов классов находятся в файлах:

- [Logger.h](#)
- [logger.cpp](#)
- [logger/Logger.cpp](#)

5.5 Структура Params

```
#include <Interface.h>
```

Открытые члены

- [Params](#) ()

Открытые атрибуты

- std::string [dbFile](#)
- std::string [logFile](#)
- int [port](#)

5.5.1 Конструктор(ы)

5.5.1.1 Params()

```
Params::Params ( ) [inline]
```

5.5.2 Данные класса

5.5.2.1 dbFile

`std::string Params::dbFile`

5.5.2.2 logFile

`std::string Params::logFile`

5.5.2.3 port

`int Params::port`

Объявления и описания членов структуры находятся в файле:

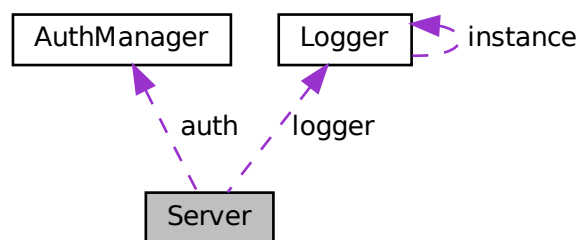
- [Interface.h](#)

5.6 Класс Server

Основной класс сетевого сервера.

```
#include <Server.h>
```

Граф связей класса Server:



Открытые члены

- [Server](#) (`int port`, `AuthManager &auth`, `Logger &logger`)
- `void run ()`
- `void stop ()`

Закрытые члены

- void `handleClient` (int clientSocket)
- std::string `receiveText` (int socket)
- bool `sendText` (int socket, const std::string &text)
- std::vector< uint32_t > `receiveVector` (int socket)
- bool `sendResult` (int socket, uint32_t result)

Закрытые данные

- int `port`
- `AuthManager` & `auth`
- `Logger` & `logger`
- bool `running`

5.6.1 Подробное описание

Основной класс сетевого сервера.

Устанавливает TCP-сокеты, ожидает подключений клиентов. Создает сессии клиентов, координирует аутентификацию и обработку данных.

5.6.2 Конструктор(ы)

5.6.2.1 Server()

```
Server::Server (  
    int port,  
    AuthManager & auth,  
    Logger & logger )
```

5.6.3 Методы

5.6.3.1 handleClient()

```
void Server::handleClient (  
    int clientSocket ) [private]
```

5.6.3.2 receiveText()

```
std::string Server::receiveText (
    int socket ) [private]
```

5.6.3.3 receiveVector()

```
std::vector< uint32_t > Server::receiveVector (
    int socket ) [private]
```

5.6.3.4 run()

```
void Server::run ( )
```

5.6.3.5 sendResult()

```
bool Server::sendResult (
    int socket,
    uint32_t result ) [private]
```

5.6.3.6 sendText()

```
bool Server::sendText (
    int socket,
    const std::string & text ) [private]
```

5.6.3.7 stop()

```
void Server::stop ( )
```

5.6.4 Данные класса

5.6.4.1 auth

[AuthManager](#)& Server::auth [private]

5.6.4.2 logger

[Logger](#)& Server::logger [private]

5.6.4.3 port

int Server::port [private]

5.6.4.4 running

bool Server::running [private]

Объявления и описания членов классов находятся в файлах:

- [Server.h](#)
- [server.cpp](#)

5.7 Класс SHA1

```
#include <SHA1.h>
```

Открытые статические члены

- static std::string [hash](#) (const std::string &input)
- static std::string [hash](#) (const std::string &salt, const std::string &password)

5.7.1 Методы

5.7.1.1 hash() [1/2]

```
std::string SHA1::hash (  
    const std::string & input ) [static]
```

5.7.1.2 hash() [2/2]

```
std::string SHA1::hash (
    const std::string & salt,
    const std::string & password ) [static]
```

Объявления и описания членов классов находятся в файлах:

- [SHA1.h](#)
- [SHA1.cpp](#)

5.8 Класс VectorProcessor

Обработчик числовых векторов.

```
#include <VectorProcessor.h>
```

Открытые статические члены

- static uint32_t [computeSumOfSquares](#) (const std::vector< uint32_t > &vector)
- static std::vector< uint32_t > [processVectors](#) (const std::vector< std::vector< uint32_t > > &vectors)

Закрытые статические члены

- static uint32_t [checkOverflow](#) (uint64_t value)

5.8.1 Подробное описание

Обработчик числовых векторов.

Вычисляет сумму квадратов элементов вектора. Обработывает переполнение (возвращает UINT32_MAX при переполнении).

5.8.2 Методы

5.8.2.1 checkOverflow()

```
uint32_t VectorProcessor::checkOverflow (
    uint64_t value ) [static], [private]
```

5.8.2.2 computeSumOfSquares()

```
uint32_t VectorProcessor::computeSumOfSquares (
    const std::vector< uint32_t > & vector ) [static]
```

5.8.2.3 processVectors()

```
std::vector< uint32_t > VectorProcessor::processVectors (
    const std::vector< std::vector< uint32_t > > & vectors ) [static]
```

Объявления и описания членов классов находятся в файлах:

- [VectorProcessor.h](#)
- [VectorProcessor.cpp](#)

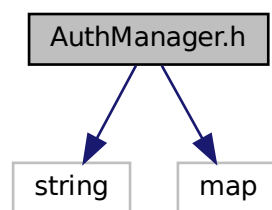
Глава 6

Файлы

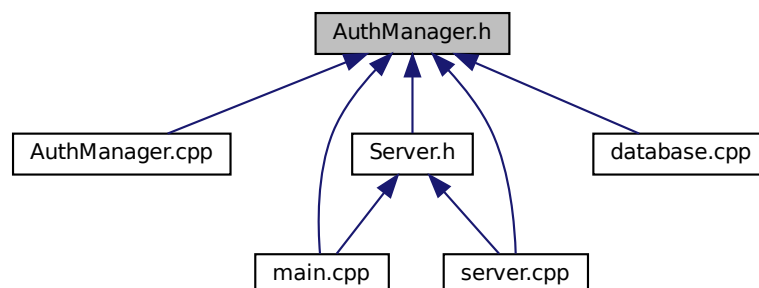
6.1 Файл AuthManager.h

```
#include <string>  
#include <map>
```

Граф включаемых заголовочных файлов для AuthManager.h:



Граф файлов, в которые включается этот файл:



Классы

- class `AuthManager`
Менеджер аутентификации пользователей.

6.2 AuthManager.h

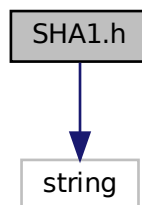
См. документацию.

```
1
8 #ifndef AUTHMANAGER_H
9 #define AUTHMANAGER_H
10
11 #include <string>
12 #include <map>
13
14 class AuthManager {
15 private:
16     std::map<std::string, std::string> users;
17
18     std::string padZeros(const std::string& str, size_t length);
19     std::string computeSHA1(const std::string& salt, const std::string& password);
20
21 public:
22     // Только то, что используется в тестах и сервере
23     AuthManager() = default;
24     ~AuthManager() = default;
25
26     bool loadUserDatabase(const std::string& filename);
27     bool userExists(const std::string& username);
28     bool authenticate(const std::string& username,
29                     const std::string& salt,
30                     const std::string& clientHash);
31     void addUser(const std::string& username, const std::string& passwordHash);
32     std::string generateSalt64();
33 };
34
35 #endif
```

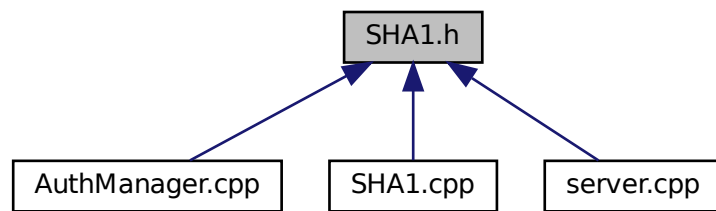
6.3 Файл SHA1.h

```
#include <string>
```

Граф включаемых заголовочных файлов для SHA1.h:



Граф файлов, в которые включается этот файл:



Классы

- class [SHA1](#)

Пространства имен

- namespace [SHA1](#)
Функции хеширования SHA-1.

6.4 SHA1.h

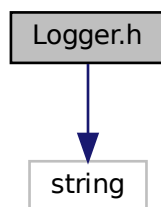
[См. документацию.](#)

```
1
8 #ifndef SHA1_H
9 #define SHA1_H
10
11 #include <string>
12
13 class SHA1 {
14 public:
15     static std::string hash(const std::string& input);
16     static std::string hash(const std::string& salt, const std::string& password);
17 };
18
19 #endif
```

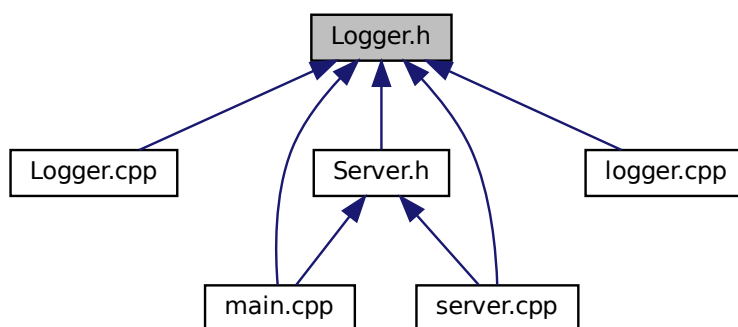
6.5 Файл Logger.h

```
#include <string>
```

Граф включаемых заголовочных файлов для Logger.h:



Граф файлов, в которые включается этот файл:



Классы

- class `Logger`
Потокобезопасный регистратор событий.

Перечисления

- enum class `LogLevel` { `INFO` , `WARNING` , `ERROR` , `CRITICAL` }

6.5.1 Перечисления

6.5.1.1 LogLevel

```
enum class LogLevel [strong]
```

Элементы перечислений

INFO	
WARNING	
ERROR	
CRITICAL	

6.6 Logger.h

См. документацию.

```

1
2 #ifndef LOGGER_H
3 #define LOGGER_H
4
5 #include <string>
6
7 enum class LogLevel {
8     INFO,
9     WARNING, // Добавляем WARNING
10    ERROR,
11    CRITICAL
12 };
13
14 class Logger {
15 private:
16     static Logger* instance; // Добавляем static declaration
17     std::string logFile; // Добавляем переменную
18
19     Logger(); // Приватный конструктор
20     Logger(const Logger&) = delete;
21     Logger& operator=(const Logger&) = delete;
22
23 public:
24     static Logger& getInstance();
25
26     bool initialize(const std::string& filename);
27     void log(LogLevel level, const std::string& message,
28             const std::string& params = "");
29     void reset();
30
31     ~Logger() = default;
32 };
33
34 #endif

```

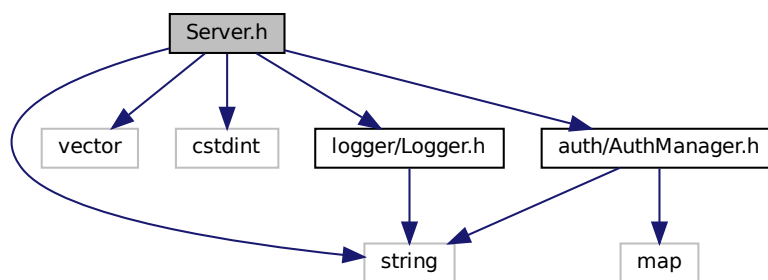
6.7 Файл Server.h

```

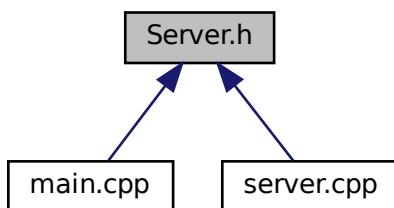
#include <string>
#include <vector>
#include <cstdint>
#include "auth/AuthManager.h"
#include "logger/Logger.h"

```

Граф включаемых заголовочных файлов для Server.h:



Граф файлов, в которые включается этот файл:



Классы

- class [Server](#)

Основной класс сетевого сервера.

6.8 Server.h

[См. документацию.](#)

```

1
8 #ifndef SERVER_H
9 #define SERVER_H
10
11 #include <string>
12 #include <vector>
13 #include <cstdlib>
14 #include "auth/AuthManager.h"
15 #include "logger/Logger.h"
16
17 class Server {
18 private:
19     int port;
20     AuthManager& auth;
21     Logger& logger;
22     bool running;

```

```

23
24 public:
25     Server(int port, AuthManager& auth, Logger& logger);
26     void run(); // Должен быть публичным!
27     void stop();
28
29 private:
30     void handleClient(int clientSocket);
31     std::string receiveText(int socket);
32     bool sendText(int socket, const std::string& text);
33     std::vector<uint32_t> receiveVector(int socket);
34     bool sendResult(int socket, uint32_t result);
35 };
36
37 #endif

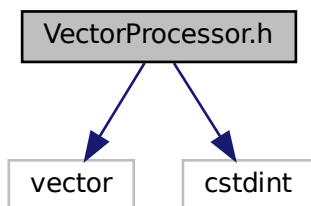
```

6.9 Файл VectorProcessor.h

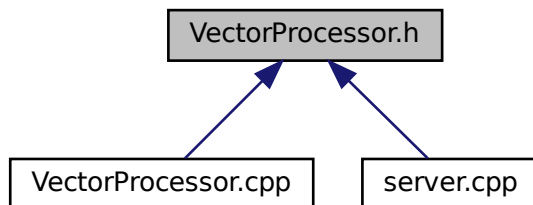
```
#include <vector>
```

```
#include <cstdint>
```

Граф включаемых заголовочных файлов для VectorProcessor.h:



Граф файлов, в которые включается этот файл:



Классы

- class [VectorProcessor](#)
Обработчик числовых векторов.

6.10 VectorProcessor.h

См. документацию.

```

1
8 #ifndef VECTORPROCESSOR_H
9 #define VECTORPROCESSOR_H
10
11 #include <vector>
12 #include <stdint>
13
14 class VectorProcessor {
15 public:
16     // Сумма квадратов (по ТЗ)
17     static uint32_t computeSumOfSquares(const std::vector<uint32_t>& vector);
18
19     // Обработка нескольких векторов
20     static std::vector<uint32_t> processVectors(
21         const std::vector<std::vector<uint32_t>>& vectors);
22
23 private:
24     static uint32_t checkOverflow(uint64_t value);
25 };
26
27 #endif

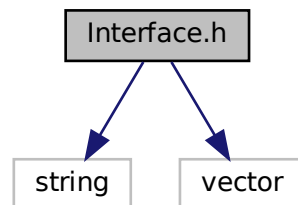
```

6.11 Файл Interface.h

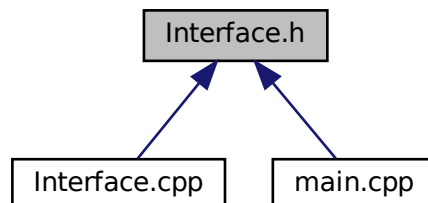
```
#include <string>
```

```
#include <vector>
```

Граф включаемых заголовочных файлов для Interface.h:



Граф файлов, в которые включается этот файл:



Классы

- struct [Params](#)
- class [Interface](#)

Парсер параметров командной строки.

6.12 Interface.h

См. документацию.

```

1
2 #ifndef INTERFACE_H
3 #define INTERFACE_H
4
5 #include <string>
6 #include <vector>
7
8 // Структура для хранения параметров командной строки
9 struct Params {
10     std::string dbFile;    // Файл базы данных пользователей
11     std::string logFile;   // Файл журнала
12     int port;             // Порт сервера
13
14     Params() : dbFile("/etc/vcalc.conf"), logFile("/var/log/vcalc.log"), port(33333) {}
15 };
16
17 // Класс для разбора и валидации параметров командной строки
18 class Interface {
19 private:
20     Params params;
21     bool helpRequested;
22     bool versionRequested;
23     std::vector<std::string> parseErrors;
24
25     // Парсинг опций командной строки с использованием getopt_long
26     void parseOptions(int argc, char* argv[]);
27
28     // Валидация разобранных параметров
29     bool validateParams();
30
31     // Проверка валидности номера порта
32     bool validatePort(int port) const;
33
34     // Проверка существования файла
35     bool checkFileExists(const std::string& filename) const;
36
37     // Проверка возможности записи в файл
38     bool checkFileWritable(const std::string& filename) const;
39
40 public:
41     // Конструктор по умолчанию
42     Interface();
43
44     // Разбор аргументов командной строки
45     bool parse(int argc, char* argv[]);
46
47     // Получение разобранных параметров
48     Params getParams() const;
49
50     // Проверка запроса справки
51     bool isHelpRequested() const;
52
53     // Проверка запроса версии
54     bool isVersionRequested() const;
55
56     // Получение ошибок парсинга
57     std::vector<std::string> getParseErrors() const;
58
59     // Вывод справки по использованию программы
60     void printHelp() const;
61
62     // Вывод информации о версии программы
63     void printVersion() const;
64 };
65
66 #endif // INTERFACE_H

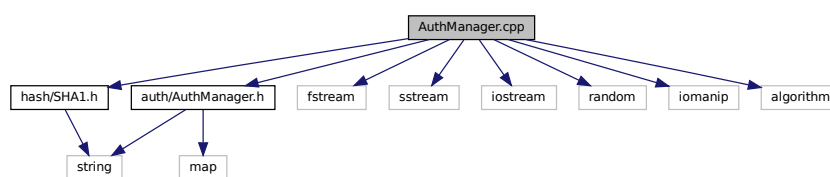
```

6.13 Файл AuthManager.cpp

Реализация менеджера аутентификации.

```
#include "auth/AuthManager.h"
#include "hash/SHA1.h"
#include <fstream>
#include <sstream>
#include <iostream>
#include <random>
#include <iomanip>
#include <algorithm>
```

Граф включаемых заголовочных файлов для AuthManager.cpp:



6.13.1 Подробное описание

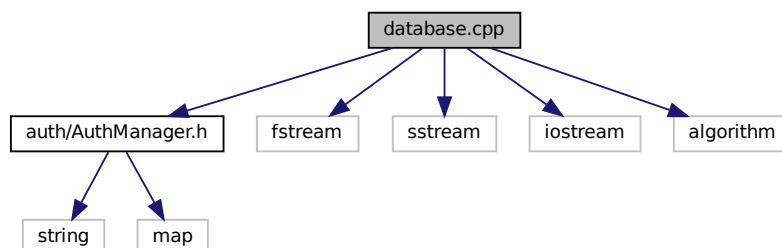
Реализация менеджера аутентификации.

Загрузка базы пользователей, проверка логинов/паролей, генерация и проверка хешей SHA-1.

6.14 Файл database.cpp

```
#include "auth/AuthManager.h"
#include <fstream>
#include <sstream>
#include <iostream>
#include <algorithm>
```

Граф включаемых заголовочных файлов для database.cpp:



Классы

- class [Database](#)

Функции

- bool [load_user_database](#) (const std::string &filename, std::map< std::string, std::string > &users)
- bool [verify_user](#) (const std::map< std::string, std::string > &users, const std::string &user, const std::string &hash)
- size_t [count_users](#) (const std::map< std::string, std::string > &users)
- void [add_user_to_db](#) (std::map< std::string, std::string > &users, const std::string &user, const std::string &hash)

6.14.1 Функции

6.14.1.1 add_user_to_db()

```
void add_user_to_db (
    std::map< std::string, std::string > & users,
    const std::string & user,
    const std::string & hash )
```

6.14.1.2 count_users()

```
size_t count_users (
    const std::map< std::string, std::string > & users )
```

6.14.1.3 load_user_database()

```
bool load_user_database (
    const std::string & filename,
    std::map< std::string, std::string > & users )
```

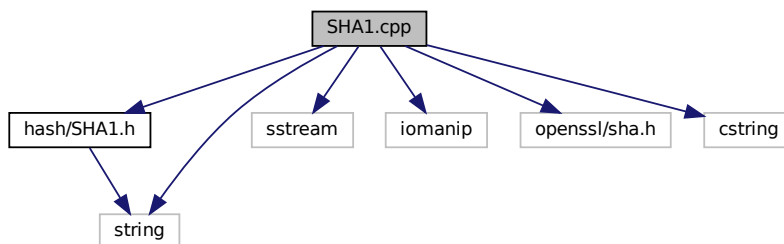
6.14.1.4 verify_user()

```
bool verify_user (
    const std::map< std::string, std::string > & users,
    const std::string & user,
    const std::string & hash )
```

6.15 Файл SHA1.cpp

```
#include "hash/SHA1.h"  
#include <sstream>  
#include <iomanip>  
#include <string>  
#include <openssl/sha.h>  
#include <cstring>
```

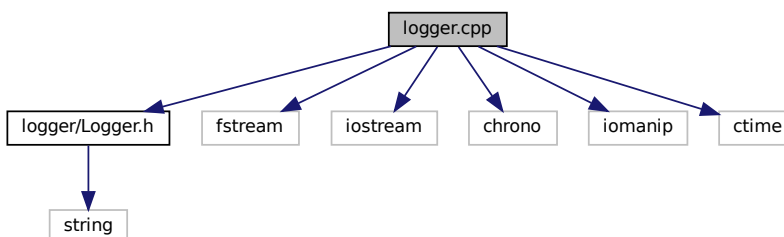
Граф включаемых заголовочных файлов для SHA1.cpp:



6.16 Файл logger.cpp

```
#include "logger/Logger.h"  
#include <fstream>  
#include <iostream>  
#include <chrono>  
#include <iomanip>  
#include <ctime>
```

Граф включаемых заголовочных файлов для logger.cpp:

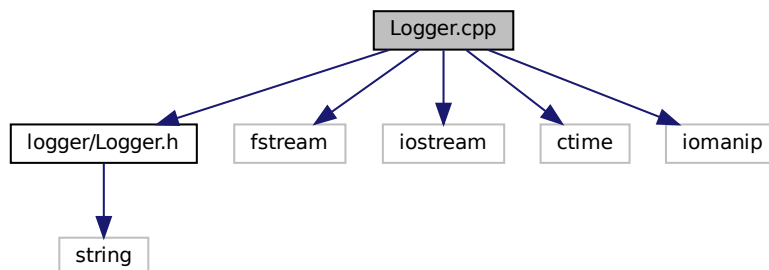


6.17 Файл Logger.cpp

Реализация логгера.

```
#include "logger/Logger.h"  
#include <fstream>  
#include <iostream>  
#include <ctime>  
#include <iomanip>
```

Граф включаемых заголовочных файлов для logger/Logger.cpp:



6.17.1 Подробное описание

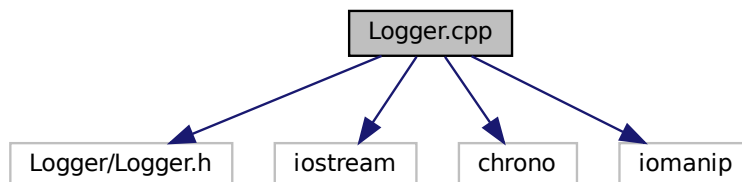
Реализация логгера.

Запись сообщений в файл с временными метками. Потокбезопасная реализация, уровни логирования.

6.18 Файл Logger.cpp

```
#include "Logger/Logger.h"  
#include <iostream>  
#include <chrono>  
#include <iomanip>
```

Граф включаемых заголовочных файлов для Logger/Logger.cpp:

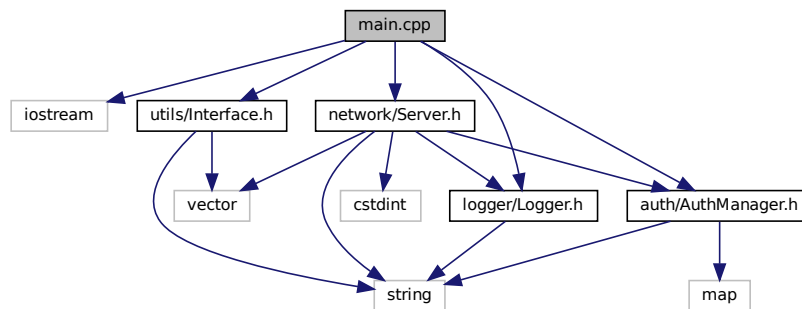


6.19 Файл main.cpp

Главная функция сервера.

```
#include <iostream>
#include "utils/Interface.h"
#include "logger/Logger.h"
#include "auth/AuthManager.h"
#include "network/Server.h"
```

Граф включаемых заголовочных файлов для main.cpp:



Функции

- int `main` (int argc, char *argv[])

6.19.1 Подробное описание

Главная функция сервера.

Точка входа в программу. Парсит аргументы командной строки, инициализирует и запускает сервер. Обработывает исключения.

6.19.2 Функции

6.19.2.1 main()

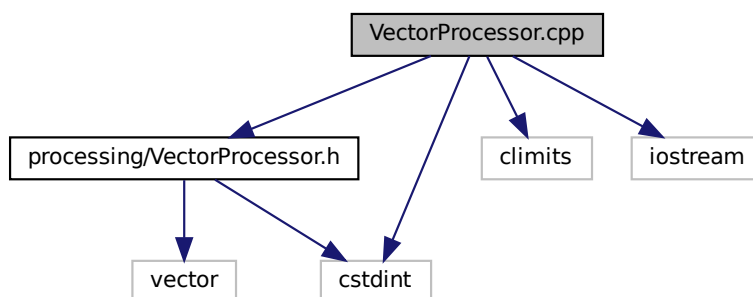
```
int main (
    int argc,
    char * argv[] )
```

6.20 Файл VectorProcessor.cpp

Реализация обработчика векторов.

```
#include "processing/VectorProcessor.h"
#include <cstdint>
#include <climits>
#include <iostream>
```

Граф включаемых заголовочных файлов для VectorProcessor.cpp:



6.20.1 Подробное описание

Реализация обработчика векторов.

Вычисление суммы квадратов с контролем переполнения. Обработка векторов в двоичном формате.

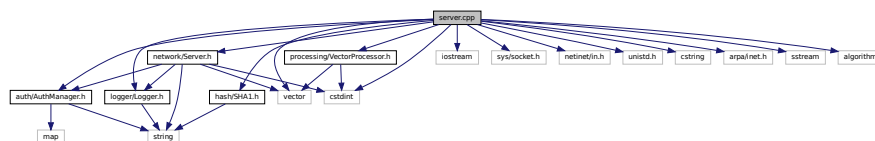
6.21 Файл server.cpp

Реализация класса [Server](#).

```
#include "network/Server.h"
#include "auth/AuthManager.h"
#include "logger/Logger.h"
#include "processing/VectorProcessor.h"
#include "hash/SHA1.h"
#include <iostream>
#include <sys/socket.h>
#include <netinet/in.h>
#include <unistd.h>
#include <cstring>
#include <vector>
#include <arpa/inet.h>
#include <sstream>
#include <algorithm>
```

```
#include <stdint>
```

Граф включаемых заголовочных файлов для server.cpp:



6.21.1 Подробное описание

Реализация класса [Server](#).

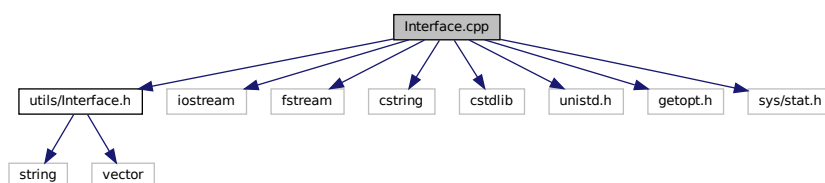
Создание TCP-сокета, привязка порта, цикл приема подключений. Управление сессиями клиентов и обработка сетевых ошибок.

6.22 Файл Interface.cpp

Реализация парсера командной строки.

```
#include "utils/Interface.h"
#include <iostream>
#include <fstream>
#include <cstring>
#include <cstdlib>
#include <unistd.h>
#include <getopt.h>
#include <sys/stat.h>
```

Граф включаемых заголовочных файлов для Interface.cpp:



Макросы

- `#define PROGRAM_VERSION "1.0.0"`
- `#define PROGRAM_NAME "vcalc_server"`

Переменные

- `static struct option long_options []`
- `const char * short_options = "hvc:l:p:"`

6.22.1 Подробное описание

Реализация парсера командной строки.

Разбор аргументов с использованием `getopt_long`. Валидация портов, проверка существования файлов.

6.22.2 Макросы

6.22.2.1 PROGRAM_NAME

```
#define PROGRAM_NAME "vcalc_server"
```

6.22.2.2 PROGRAM_VERSION

```
#define PROGRAM_VERSION "1.0.0"
```

6.22.3 Переменные

6.22.3.1 long_options

```
struct option long_options[] [static]
```

Инициализатор

```
= {  
    {"help", no_argument, 0, 'h'},  
    {"version", no_argument, 0, 'v'},  
    {"config", required_argument, 0, 'c'},  
    {"log", required_argument, 0, 'l'},  
    {"port", required_argument, 0, 'p'},  
    {0, 0, 0, 0}  
}
```

6.22.3.2 short_options

```
const char* short_options = "hvc:l:p:"
```


Предметный указатель

- ~AuthManager
 - AuthManager, 10
- ~Logger
 - Logger, 17
- add
 - Database, 12
- add_user_to_db
 - database.cpp, 37
- addUser
 - AuthManager, 10
- auth
 - Server, 22
- authenticate
 - AuthManager, 10
- AuthManager, 9
 - ~AuthManager, 10
 - addUser, 10
 - authenticate, 10
 - AuthManager, 10
 - computeSHA1, 10
 - generateSalt64, 10
 - loadUserDatabase, 10
 - padZeros, 11
 - userExists, 11
 - users, 11
- AuthManager.cpp, 36
- AuthManager.h, 27, 28
- checkFileExists
 - Interface, 14
- checkFileWritable
 - Interface, 14
- checkOverflow
 - VectorProcessor, 24
- computeSHA1
 - AuthManager, 10
- computeSumOfSquares
 - VectorProcessor, 24
- count
 - Database, 12
- count_users
 - database.cpp, 37
- CRITICAL
 - Logger.h, 31
- Database, 11
 - add, 12
 - count, 12
 - load, 12
 - users, 12
 - verify, 12
- database.cpp, 36
 - add_user_to_db, 37
 - count_users, 37
 - load_user_database, 37
 - verify_user, 37
- dbFile
 - Params, 19
- ERROR
 - Logger.h, 31
- generateSalt64
 - AuthManager, 10
- getInstance
 - Logger, 18
- getParams
 - Interface, 14
- getParseErrors
 - Interface, 14
- handleClient
 - Server, 21
- hash
 - SHA1, 23
- helpRequested
 - Interface, 16
- INFO
 - Logger.h, 31
- initialize
 - Logger, 18
- instance
 - Logger, 18
- Interface, 13
 - checkFileExists, 14
 - checkFileWritable, 14
 - getParams, 14
 - getParseErrors, 14
 - helpRequested, 16
 - Interface, 14
 - isHelpRequested, 14
 - isVersionRequested, 15
 - params, 16
 - parse, 15
 - parseErrors, 16
 - parseOptions, 15
 - printHelp, 15
 - printVersion, 15

- validateParams, 15
- validatePort, 15
- versionRequested, 16
- Interface.cpp, 42
 - long_options, 43
 - PROGRAM_NAME, 43
 - PROGRAM_VERSION, 43
 - short_options, 43
- Interface.h, 34, 35
- isHelpRequested
 - Interface, 14
- isVersionRequested
 - Interface, 15
- load
 - Database, 12
- load_user_database
 - database.cpp, 37
- loadUserDatabase
 - AuthManager, 10
- log
 - Logger, 18
- logFile
 - Logger, 19
 - Params, 20
- Logger, 16
 - ~Logger, 17
 - getInstance, 18
 - initialize, 18
 - instance, 18
 - log, 18
 - logFile, 19
 - Logger, 17
 - operator=, 18
 - reset, 18
- logger
 - Server, 23
- Logger.cpp, 38, 39
- logger.cpp, 38
- Logger.h, 30, 31
 - CRITICAL, 31
 - ERROR, 31
 - INFO, 31
 - LogLevel, 30
 - WARNING, 31
- LogLevel
 - Logger.h, 30
- long_options
 - Interface.cpp, 43
- main
 - main.cpp, 40
- main.cpp, 40
 - main, 40
- operator=
 - Logger, 18
- padZeros
 - AuthManager, 11
- Params, 19
 - dbFile, 19
 - logFile, 20
 - Params, 19
 - port, 20
- params
 - Interface, 16
- parse
 - Interface, 15
- parseErrors
 - Interface, 16
- parseOptions
 - Interface, 15
- port
 - Params, 20
 - Server, 23
- printHelp
 - Interface, 15
- printVersion
 - Interface, 15
- processVectors
 - VectorProcessor, 25
- PROGRAM_NAME
 - Interface.cpp, 43
- PROGRAM_VERSION
 - Interface.cpp, 43
- receiveText
 - Server, 21
- receiveVector
 - Server, 22
- reset
 - Logger, 18
- run
 - Server, 22
- running
 - Server, 23
- sendResult
 - Server, 22
- sendText
 - Server, 22
- Server, 20
 - auth, 22
 - handleClient, 21
 - logger, 23
 - port, 23
 - receiveText, 21
 - receiveVector, 22
 - run, 22
 - running, 23
 - sendResult, 22
 - sendText, 22
 - Server, 21
 - stop, 22
- server.cpp, 41
- Server.h, 31, 32
- SHA1, 7, 23

- hash, [23](#)
- SHA1.cpp, [38](#)
- SHA1.h, [28](#), [29](#)
- short_options
 - Interface.cpp, [43](#)
- stop
 - Server, [22](#)
- userExists
 - AuthManager, [11](#)
- users
 - AuthManager, [11](#)
 - Database, [12](#)
- validateParams
 - Interface, [15](#)
- validatePort
 - Interface, [15](#)
- VectorProcessor, [24](#)
 - checkOverflow, [24](#)
 - computeSumOfSquares, [24](#)
 - processVectors, [25](#)
- VectorProcessor.cpp, [41](#)
- VectorProcessor.h, [33](#), [34](#)
- verify
 - Database, [12](#)
- verify_user
 - database.cpp, [37](#)
- versionRequested
 - Interface, [16](#)
- WARNING
 - Logger.h, [31](#)